

Week 09

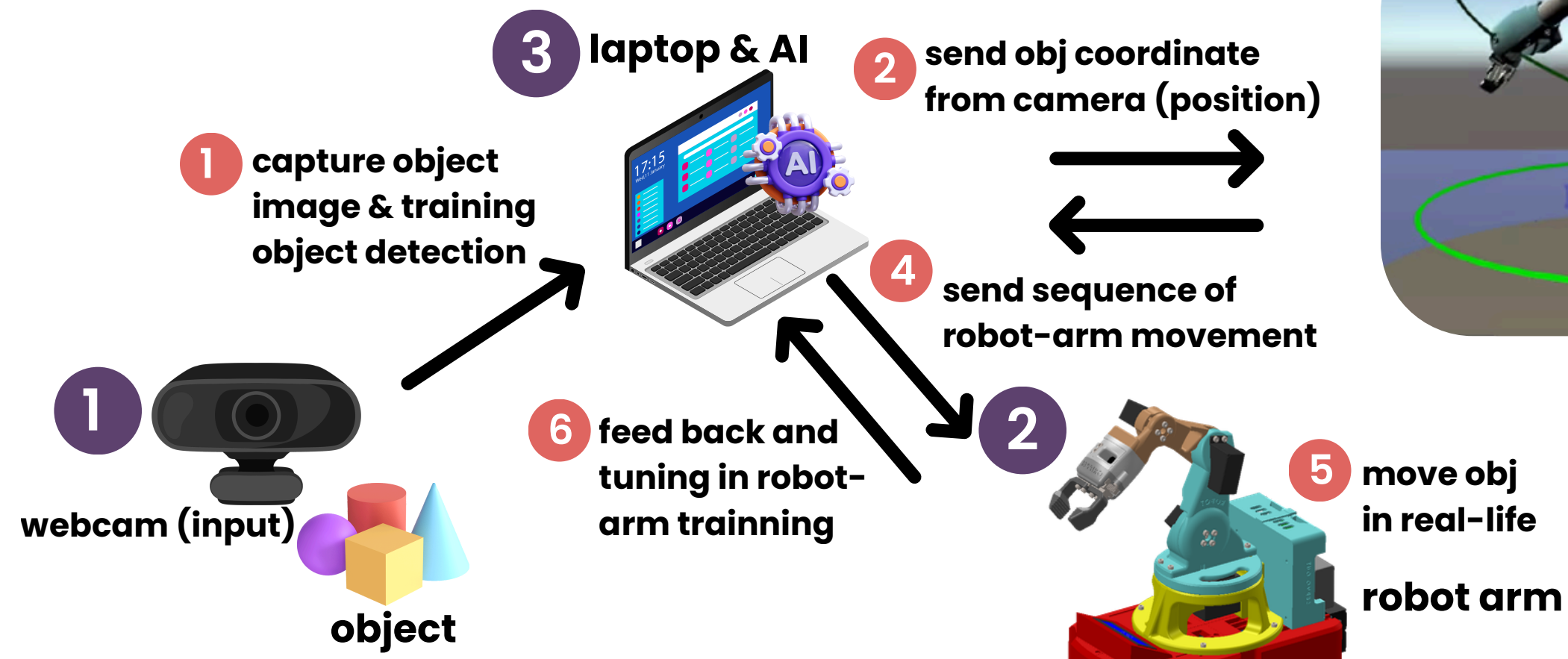
Reinforcement learning for Control RobotArm

RL for Digital Twin

Are you ready?



Content of this week



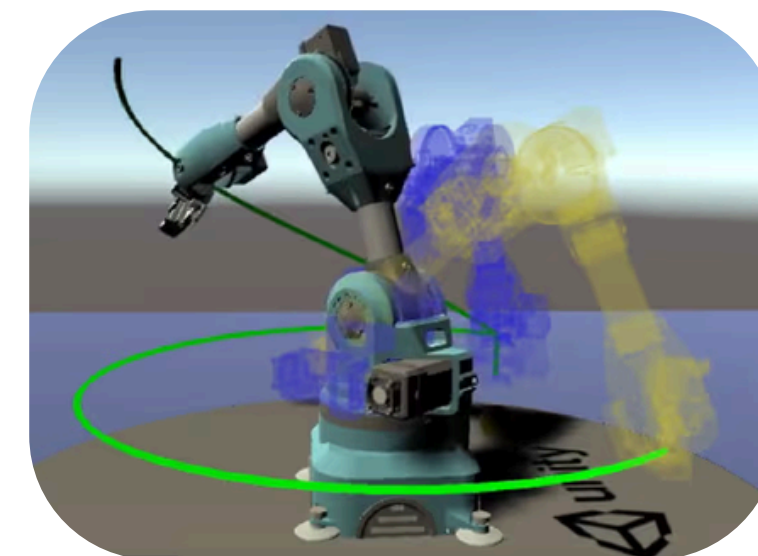
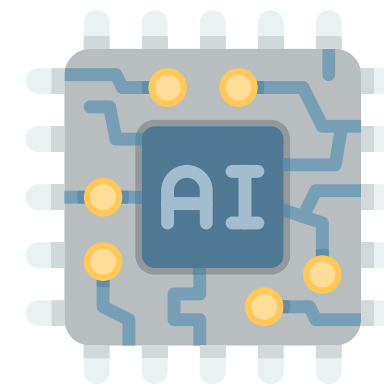
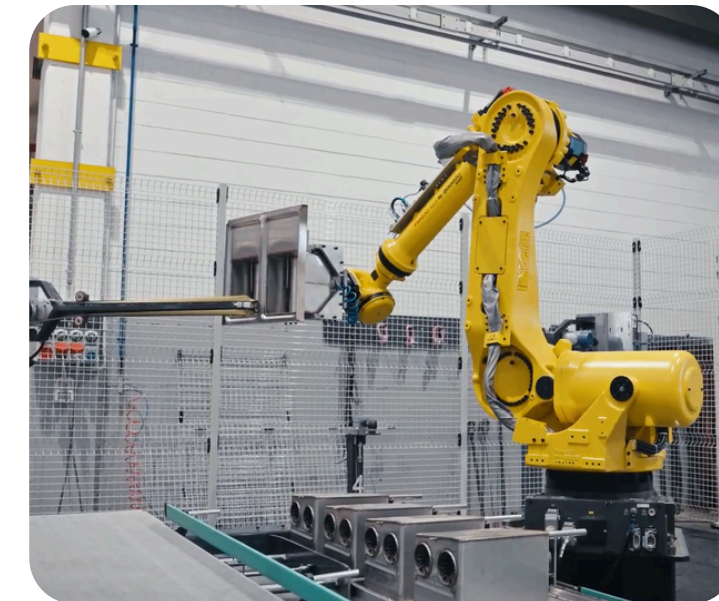
RECAP

What tool's
using for
training
automated
Robot Arm

Reinforcement learning

Action

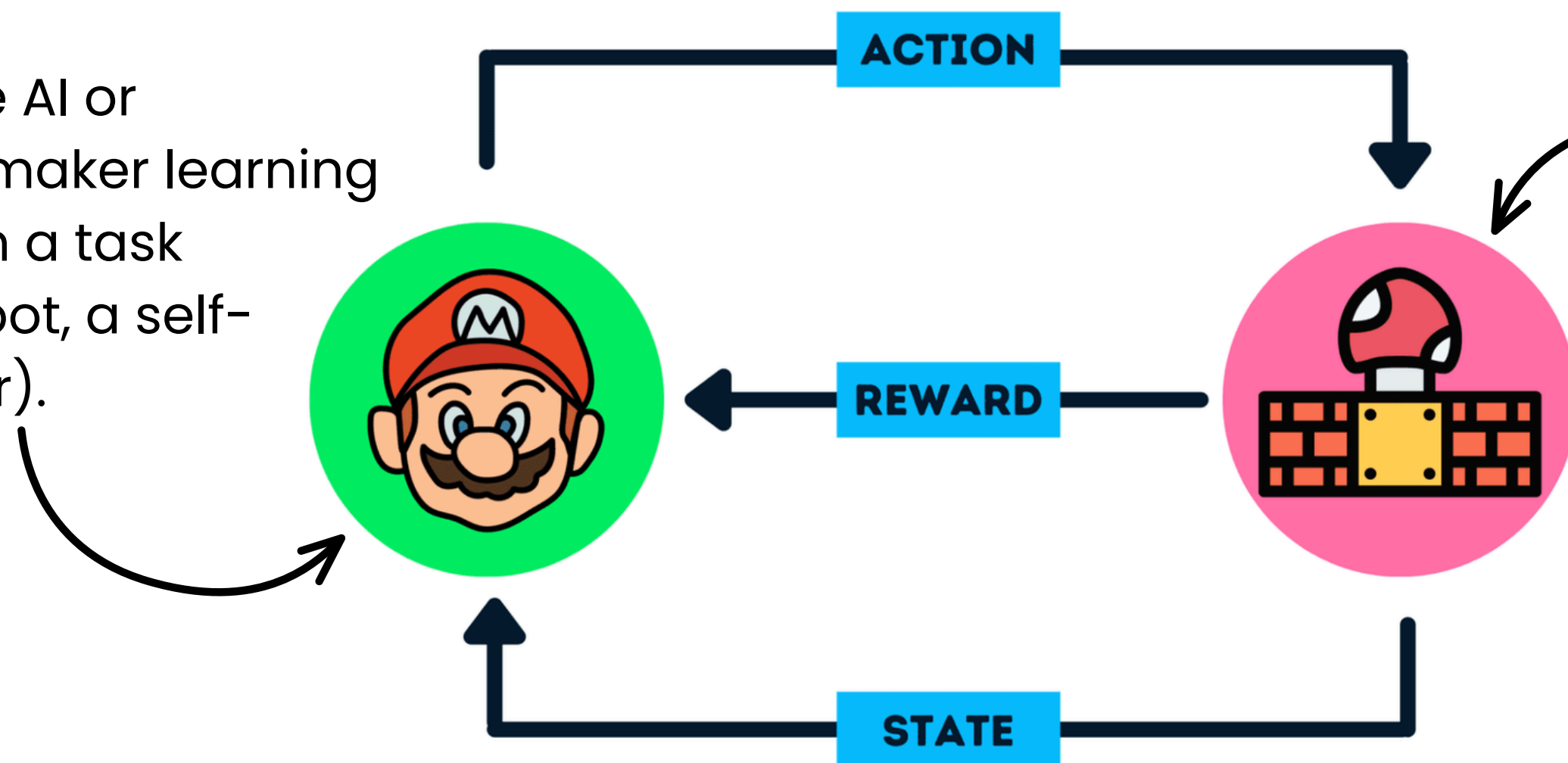
Learning



recap-component of RL

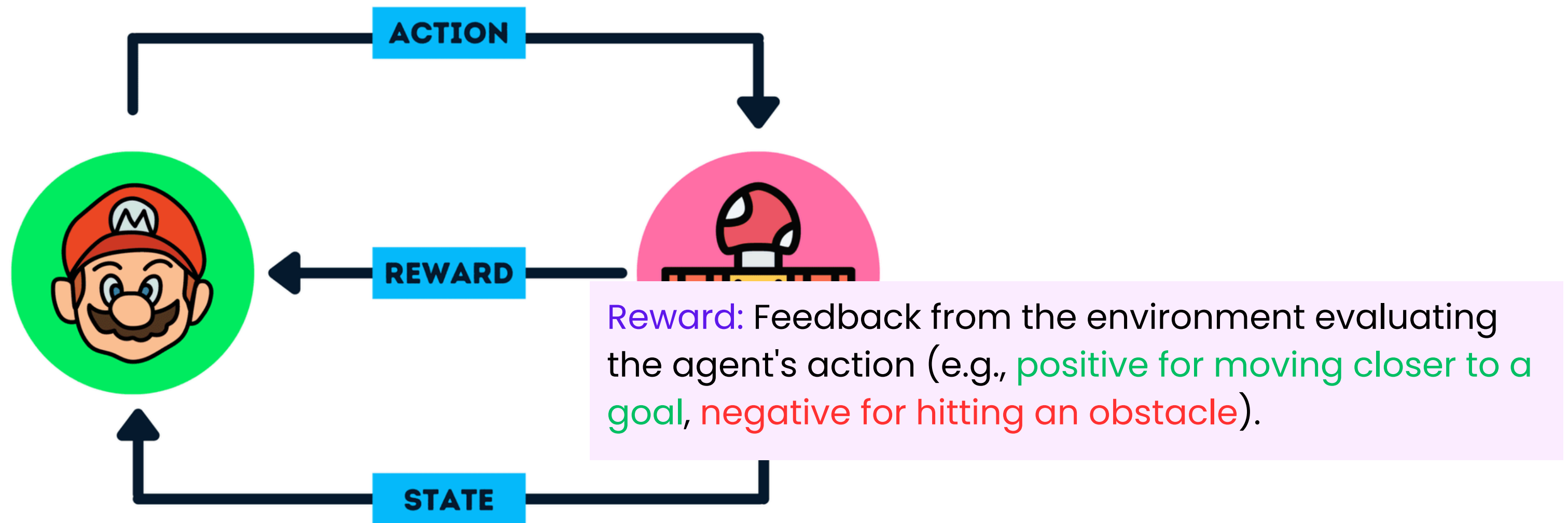
Agent: The AI or decision-maker learning to perform a task (e.g., a robot, a self-driving car).

Environment: The space or system the agent interacts with.



recap-component of RL

Action: The moves or decisions the agent can take.



State: The current condition or situation of the environment perceived by the agent.

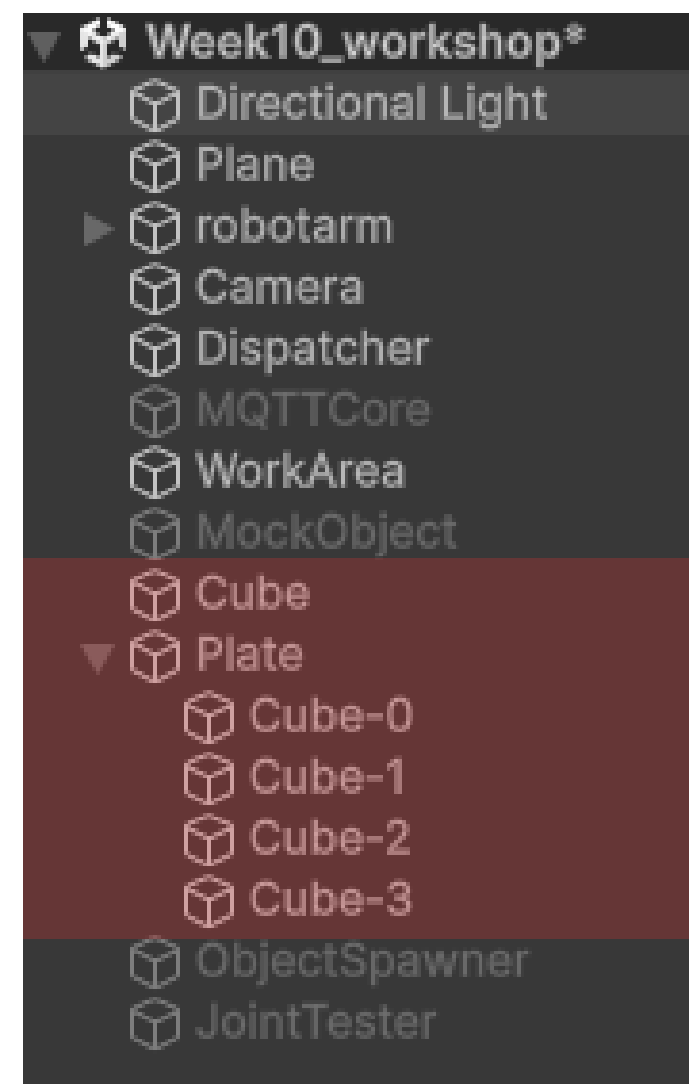
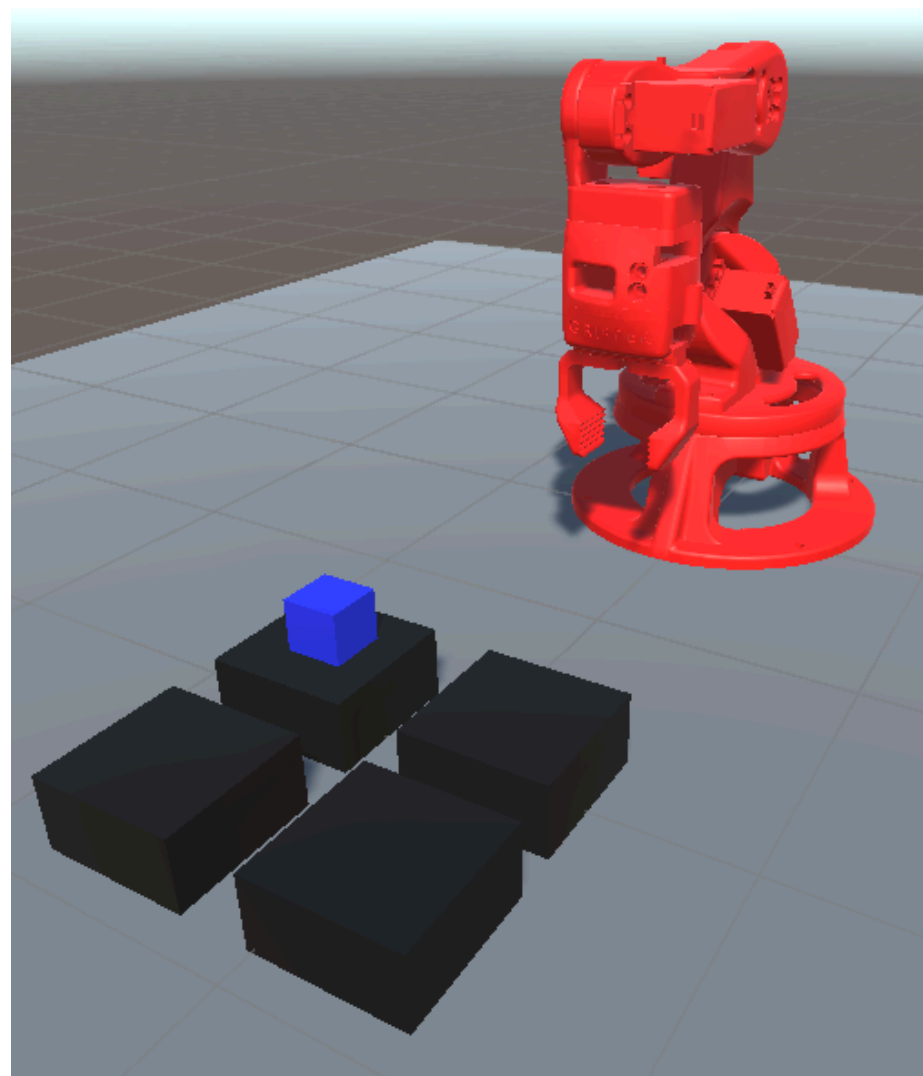
WORKSHOP

Prepare environment
Training RebotArm
unity-ml.agent

<https://github.com/idektep/DGT5/tree/week9>



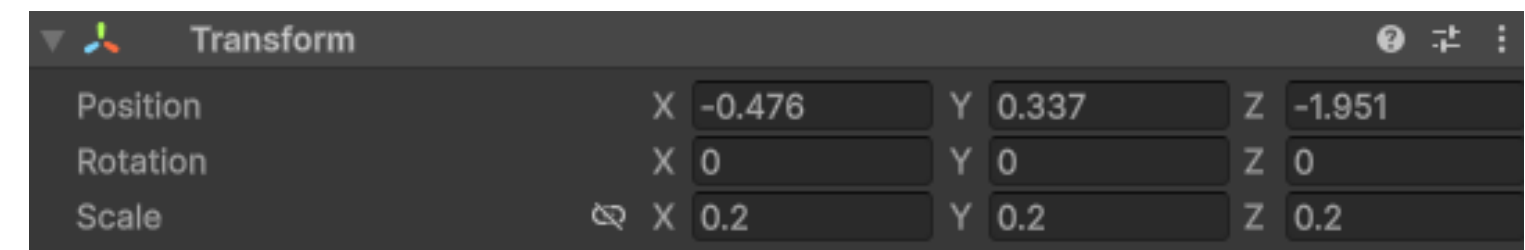
Step 1: prepare environment



STEP 1 – สร้าง object

เมนู GameObject → 3D Object → Cube → ชื่อ Cube

Transform → Scale = (1, 1, 1)



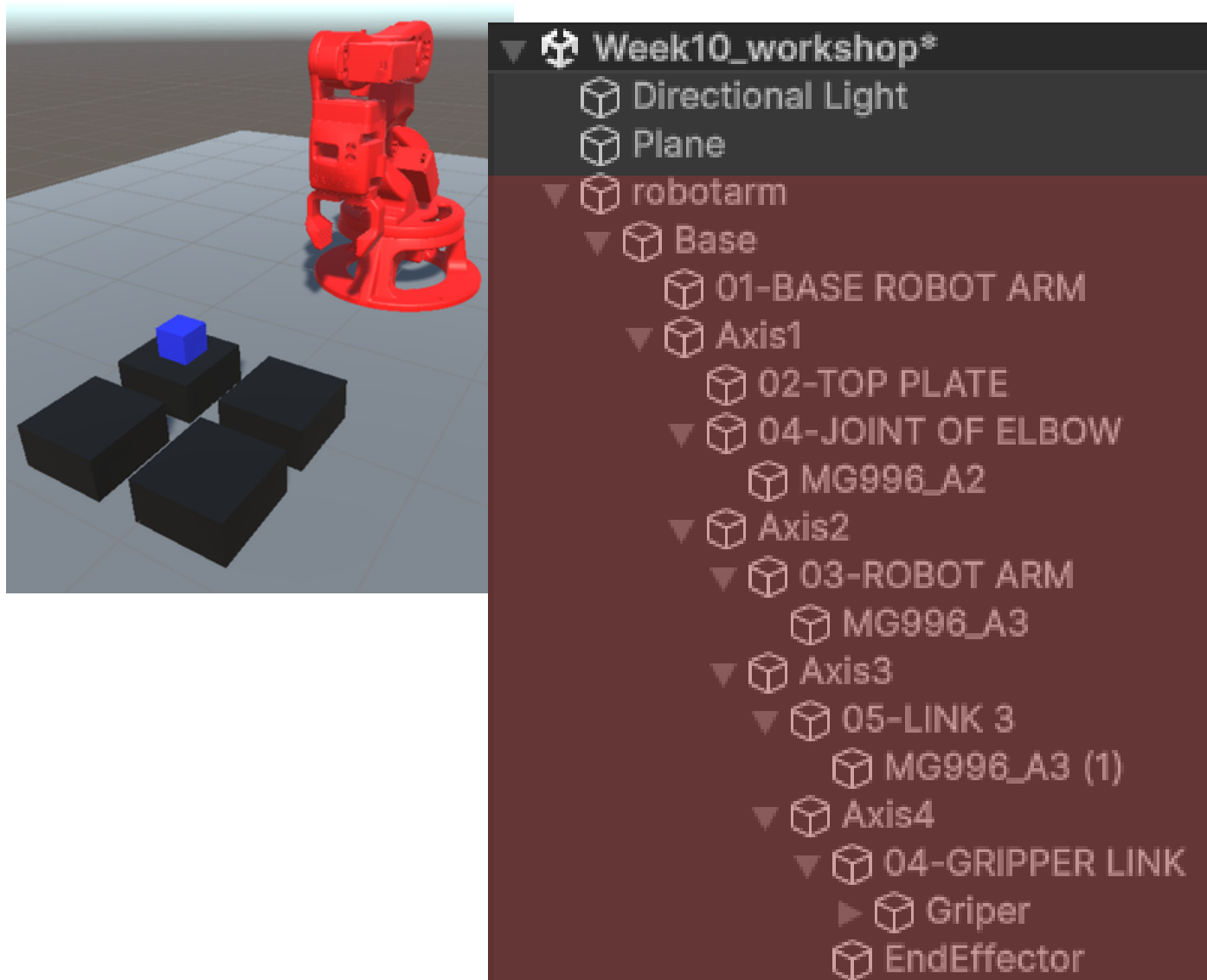
STEP 2 – สร้าง Plate (Cube-0 to Cube-4)

Create Empty → ชื่อ Plate

เมนู GameObject → 3D Object → Cube → ชื่อ Cube-0

Transform → Scale = (0.5, 0.5, 0.5)

Step 2: prepare joint RobotArm

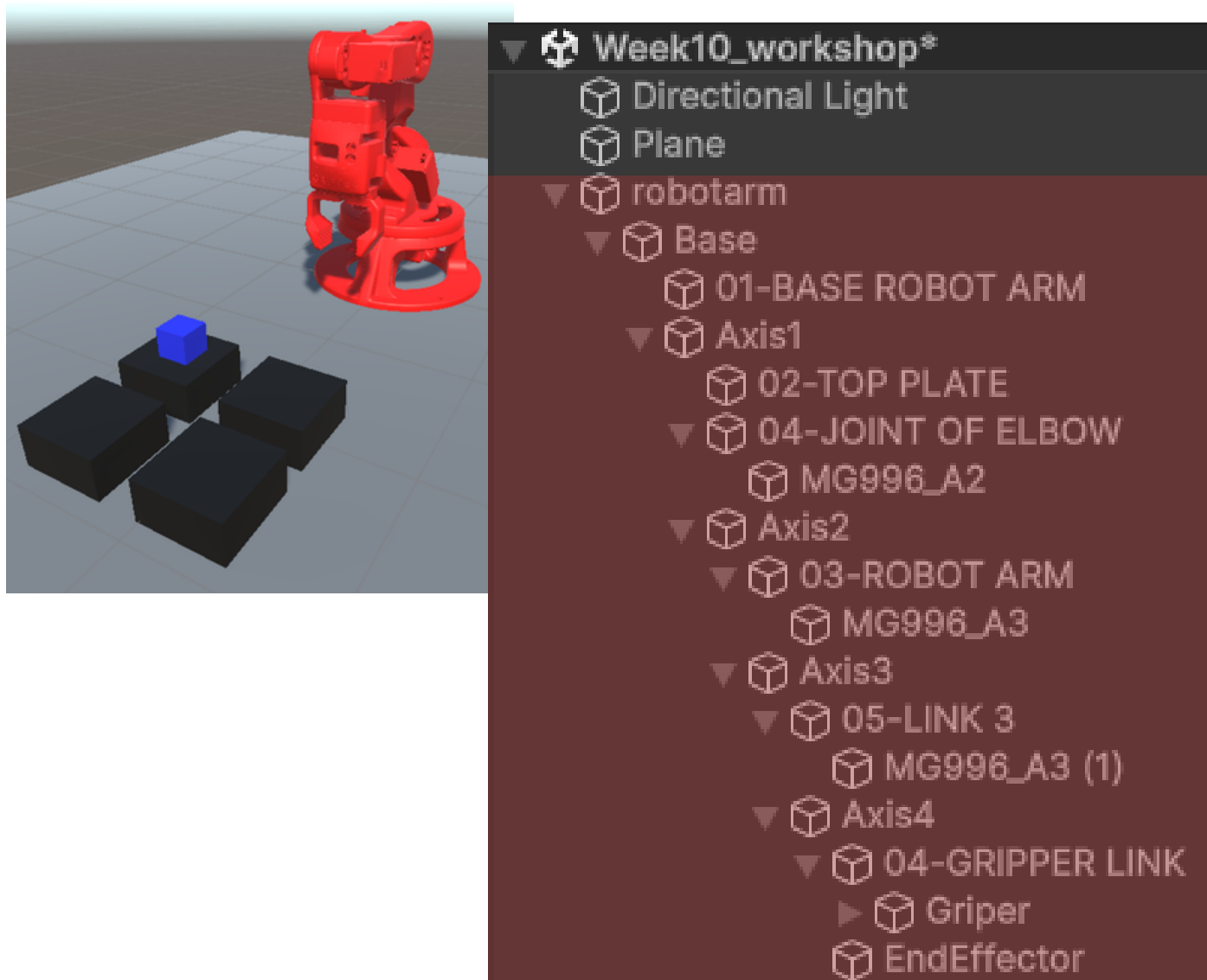


Check Transform of Axis1–Axis4

- **Axis1 rotate in ?**
- **Axis2 rotate in ?**
- **Axis3 rotate in ?**
- **Axis4 rotate in ?**

**Let's Testing and Recheck
5 mins**

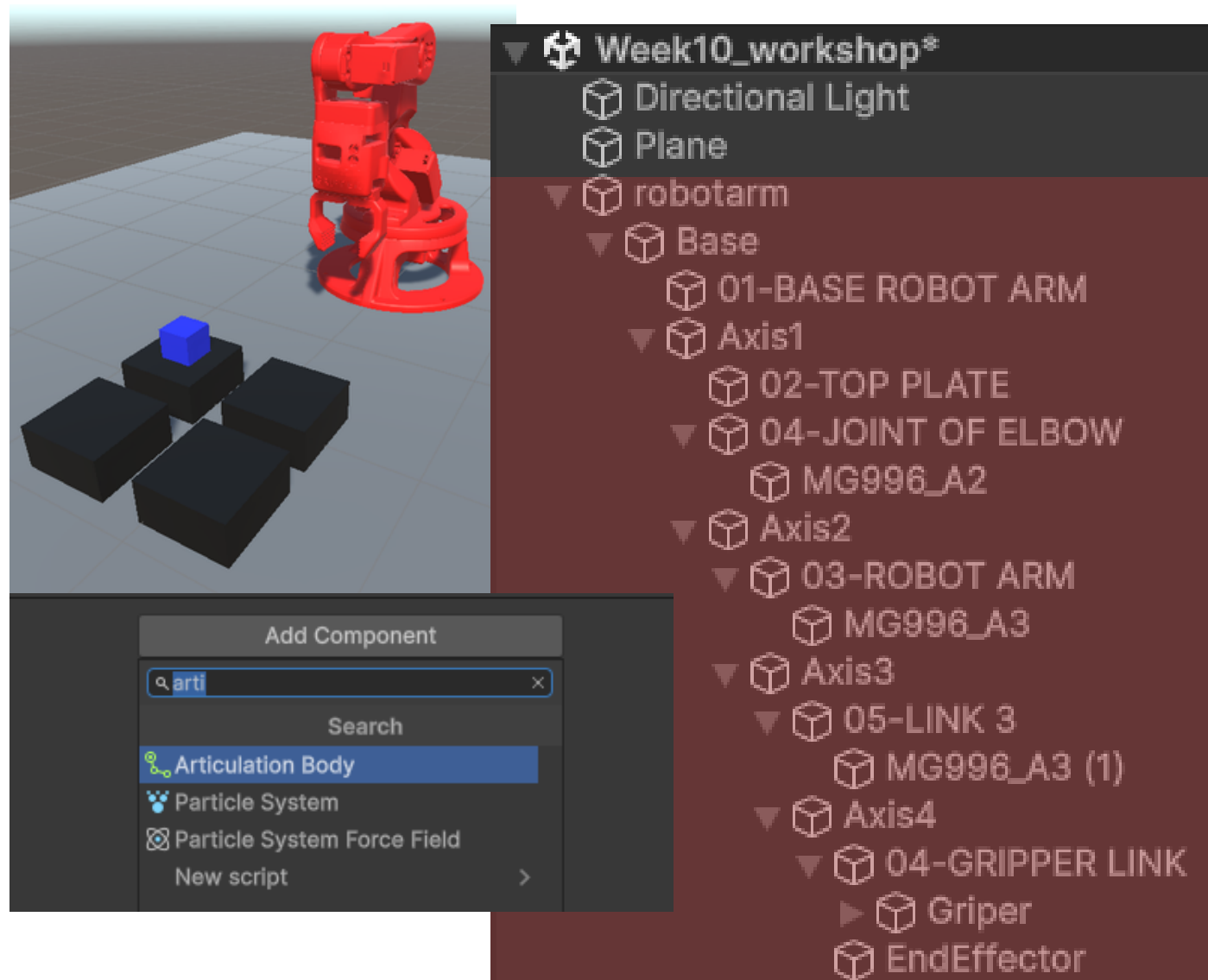
Step 2: prepare joint RobotArm



Check Transform of Axis1–Axis4

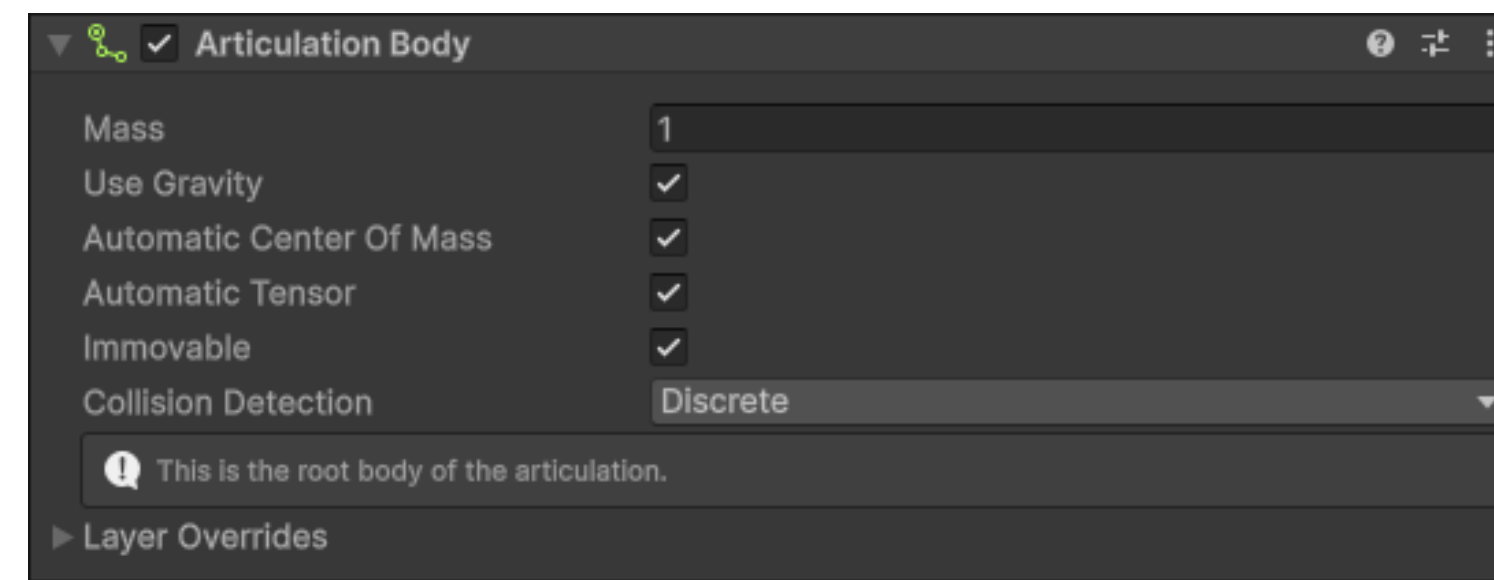
- **Axis1 rotate in Y**
- **Axis2 rotate in X**
- **Axis3 rotate in X**
- **Axis4 rotate in X**

Step 2: prepare joint RobotArm

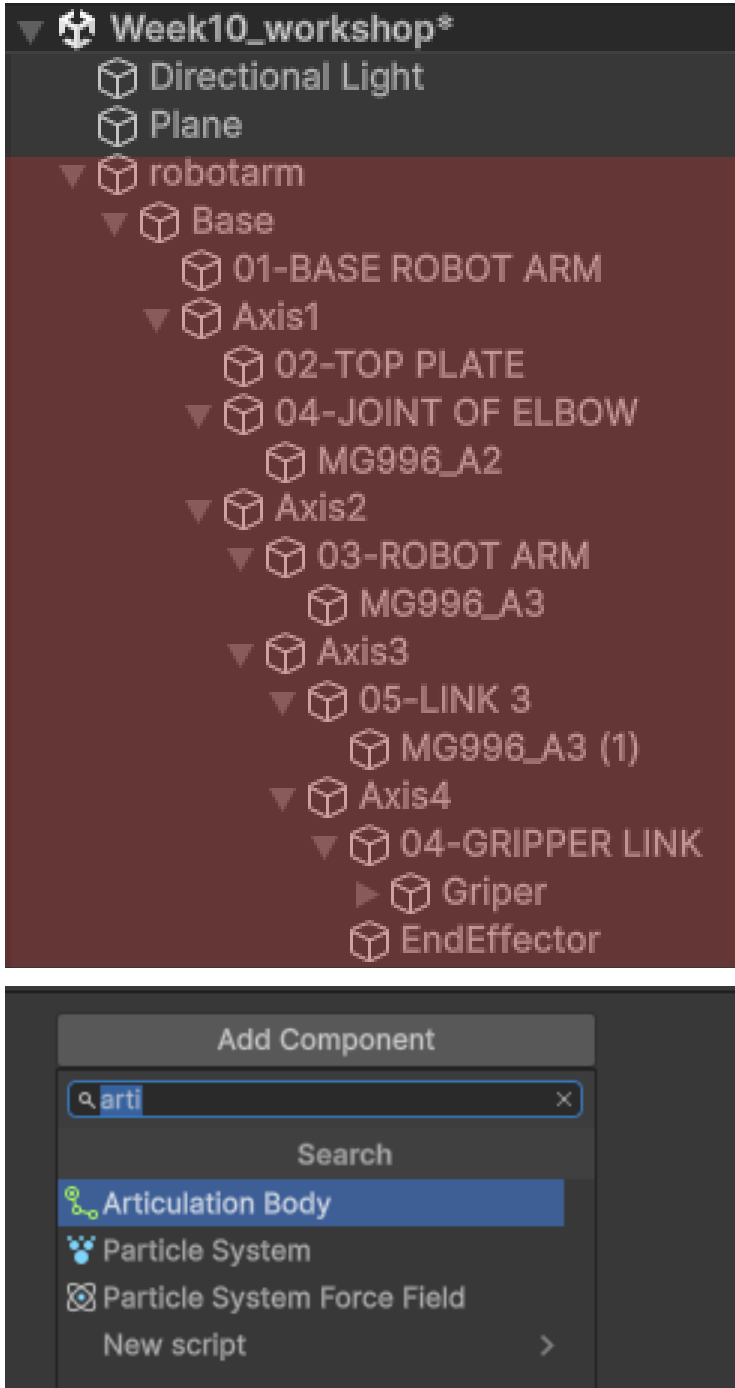


STEP – เลือก Base

Add Component → Articulation Body →
Check Immovable



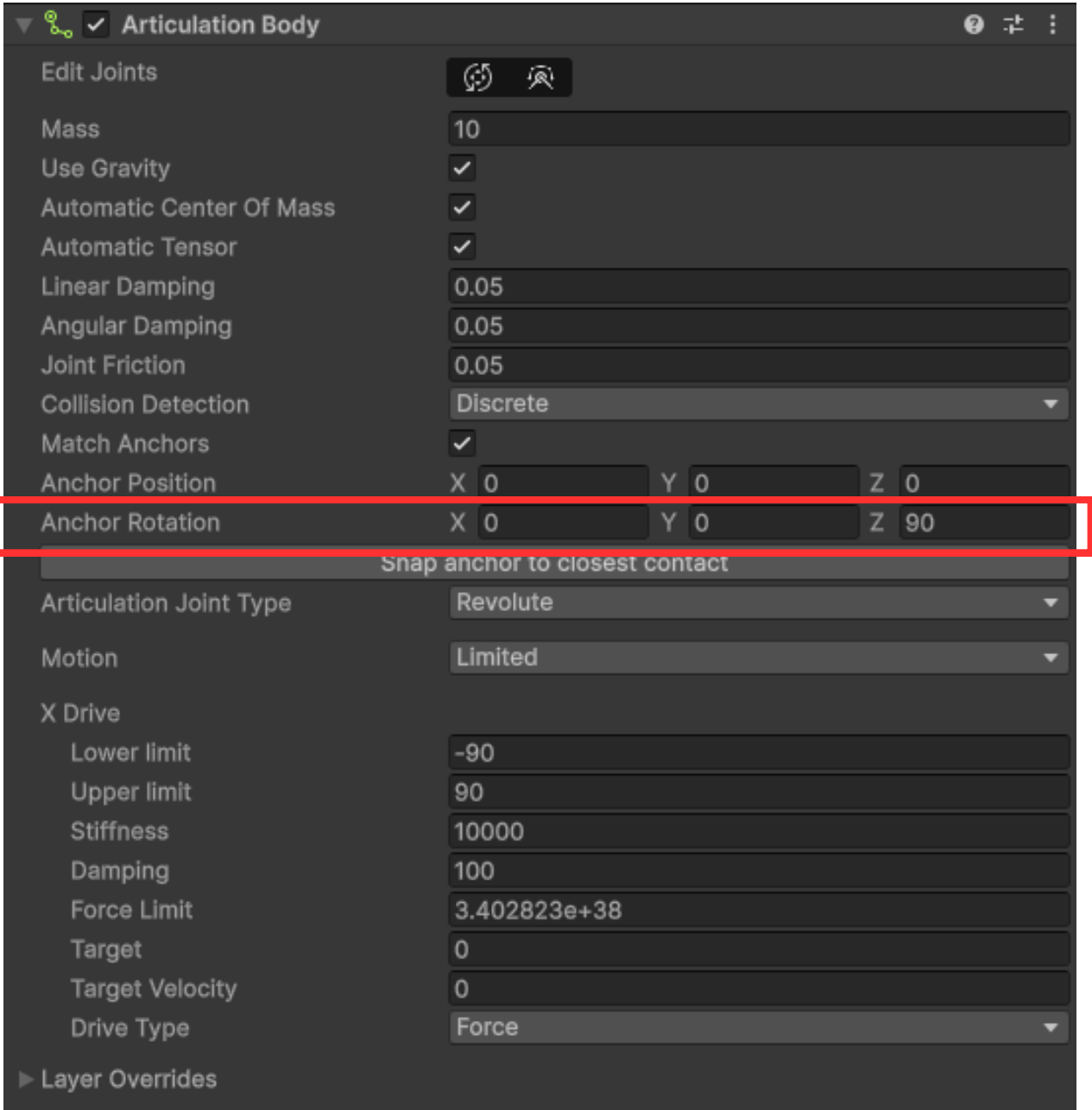
Step 2: prepare joint RobotArm



STEP – เลือก Axis1
Add Component → Articulation Body
→ Config by follow this setting

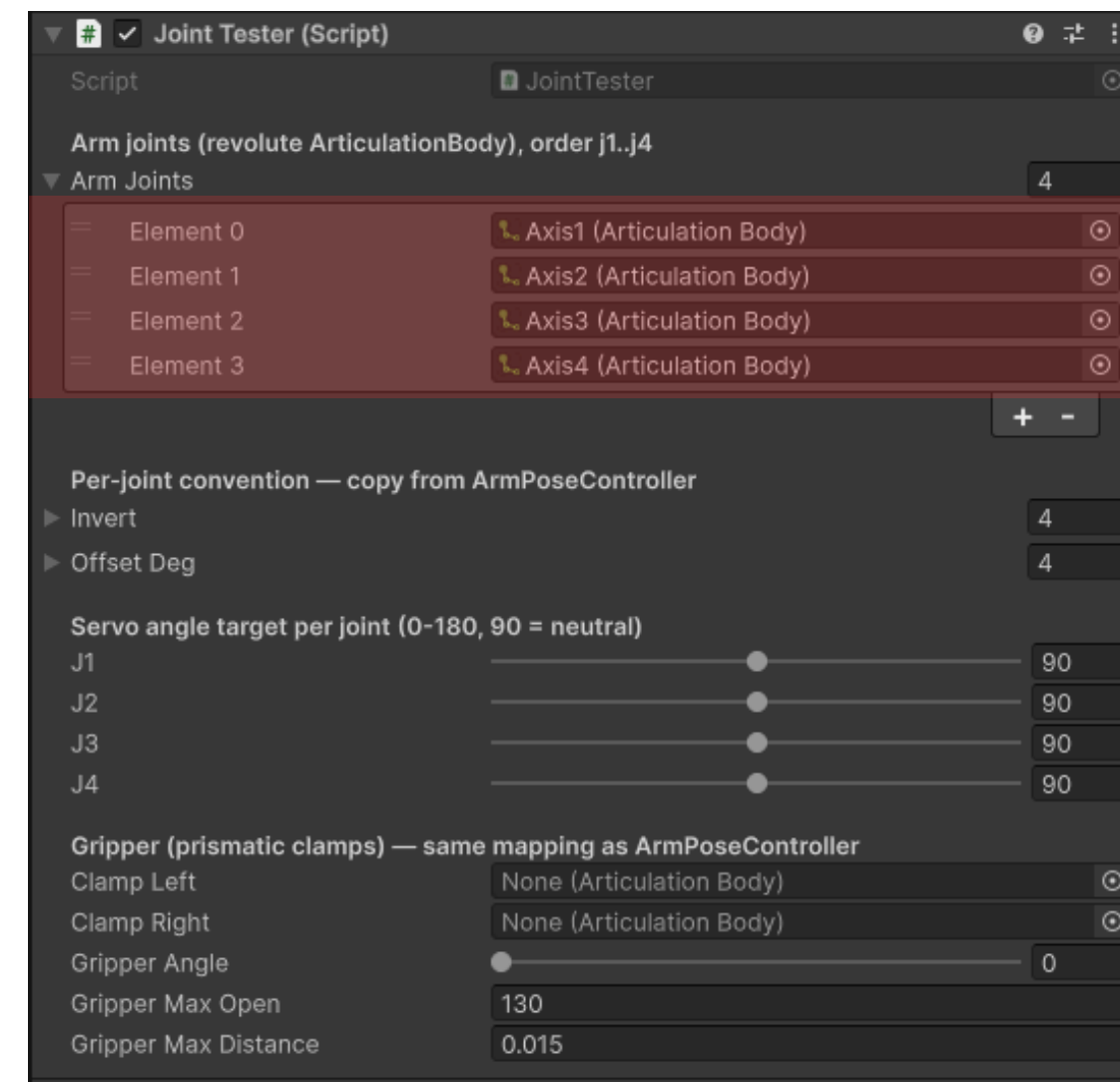
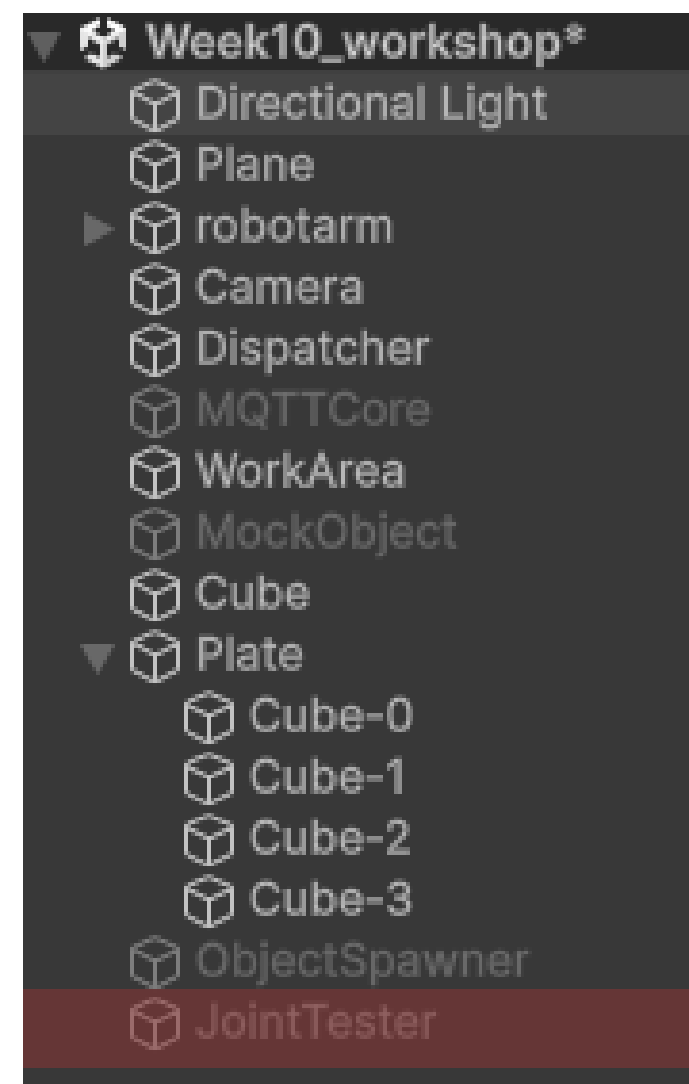
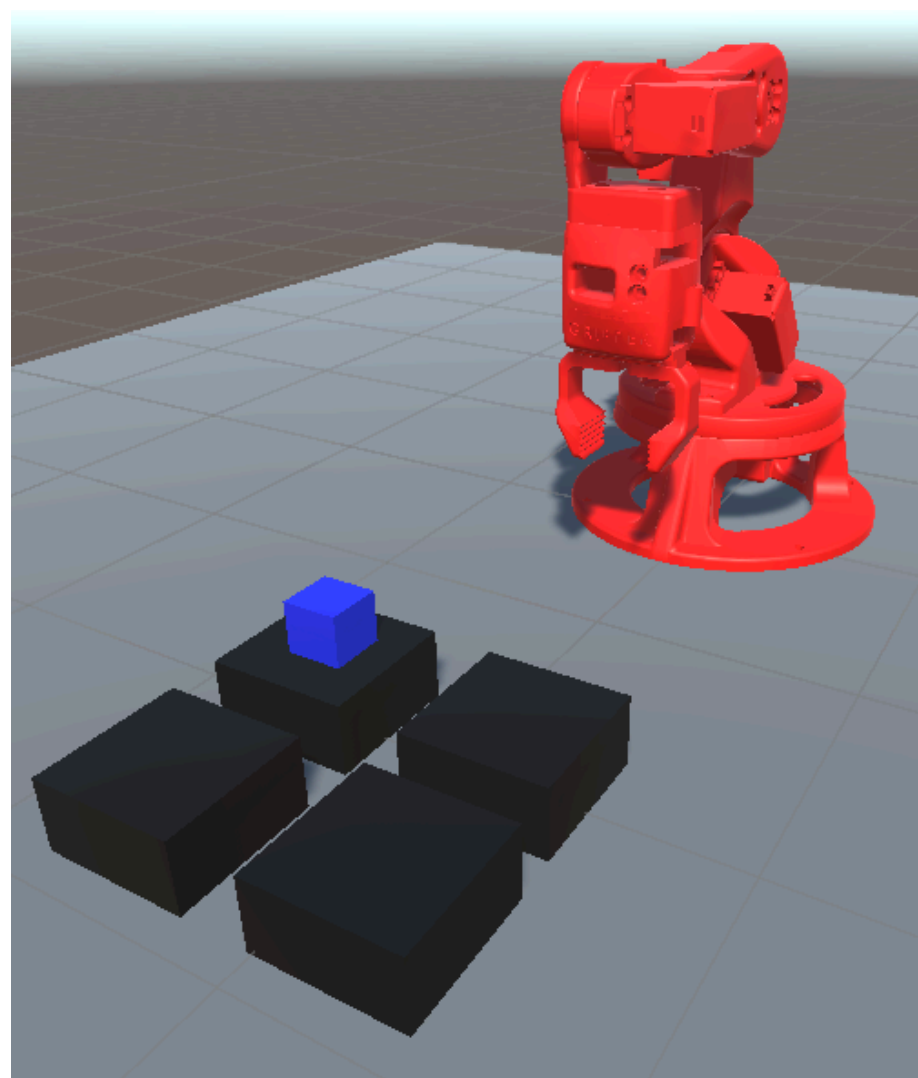
STEP – **Axis2, Axis3, Axis4**: repeat
this step of Axis1 on each

Joint	Joint Type	หมุนรอบ	Anchor Rotation (เริ่มต้น)	Lower / Upper (ใน X Drive)
Axis1	Revolute	Y	(0, 0, 90)	-90 / 90
Axis2	Revolute	X	(0, 0, 0)	-90 / 90
Axis3	Revolute	X	(0, 0, 0)	-90 / 90
Axis4	Revolute	X	(0, 0, 0)	-90 / 90



Step 2: prepare joint RobotArm

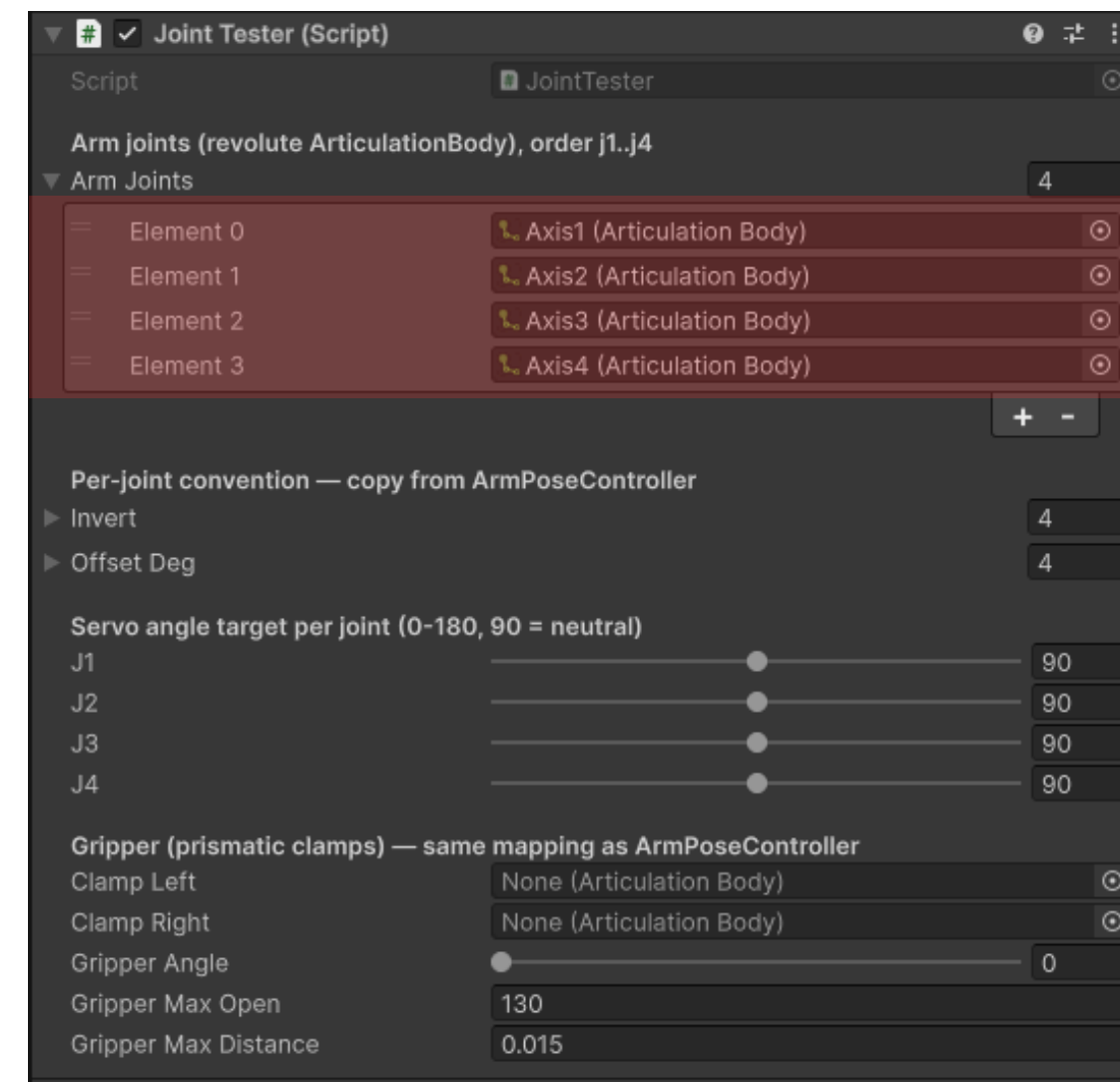
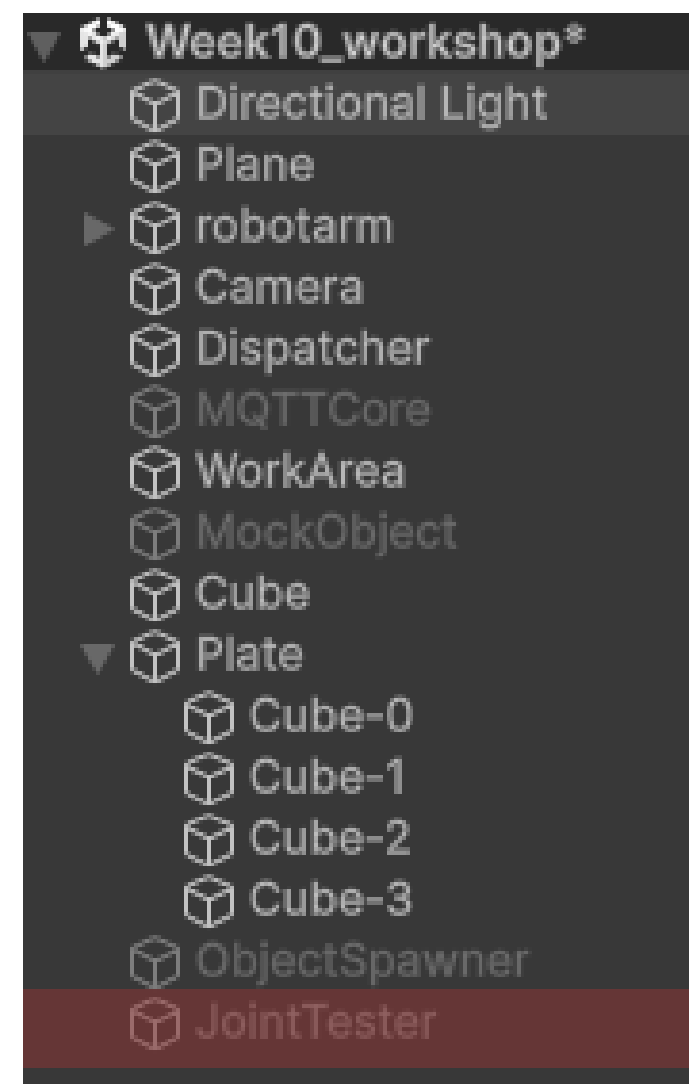
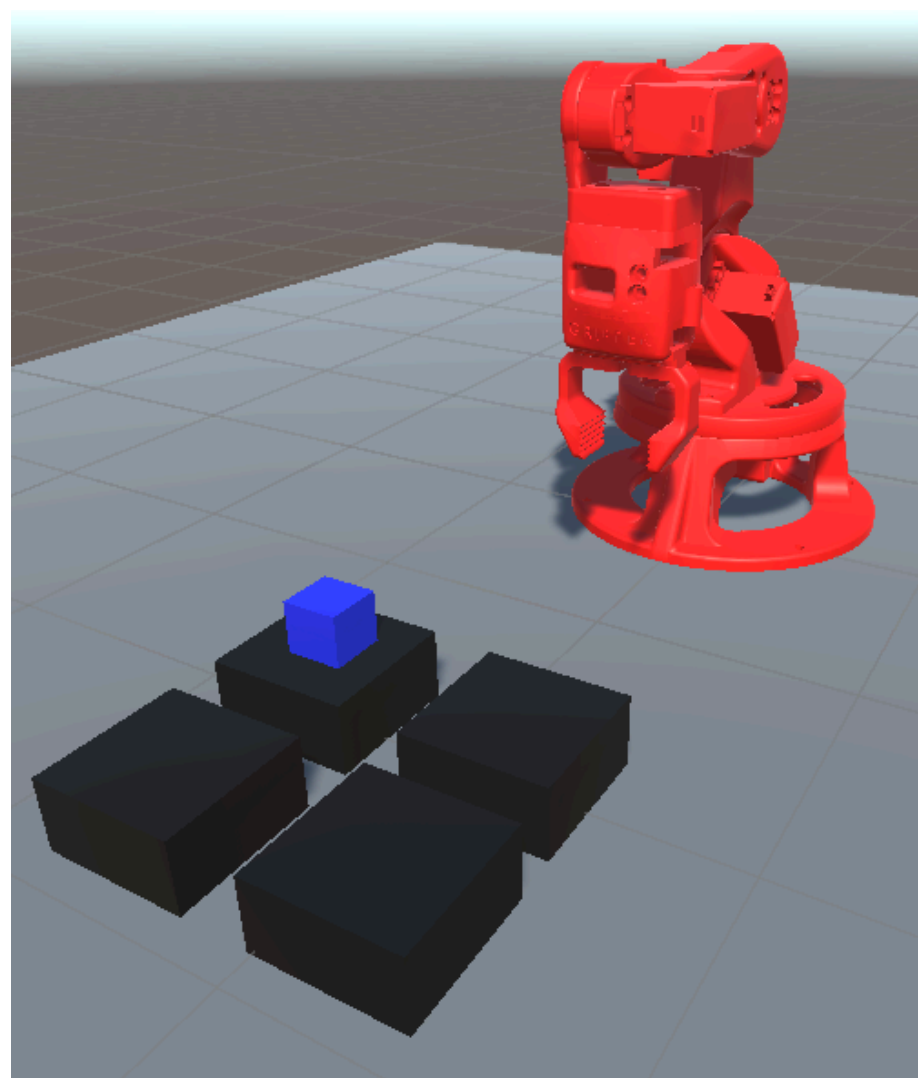
STEP 1 – Hierarchy → Create Empty → ตั้งชื่อ JointTester → Add Component → JointTester



STEP – กด play → เลือก JointTester และปรับ J1-J4 สำหรับเช็คว่ robot arm ขยับถูกต้อง

Step 2: prepare joint RobotArm

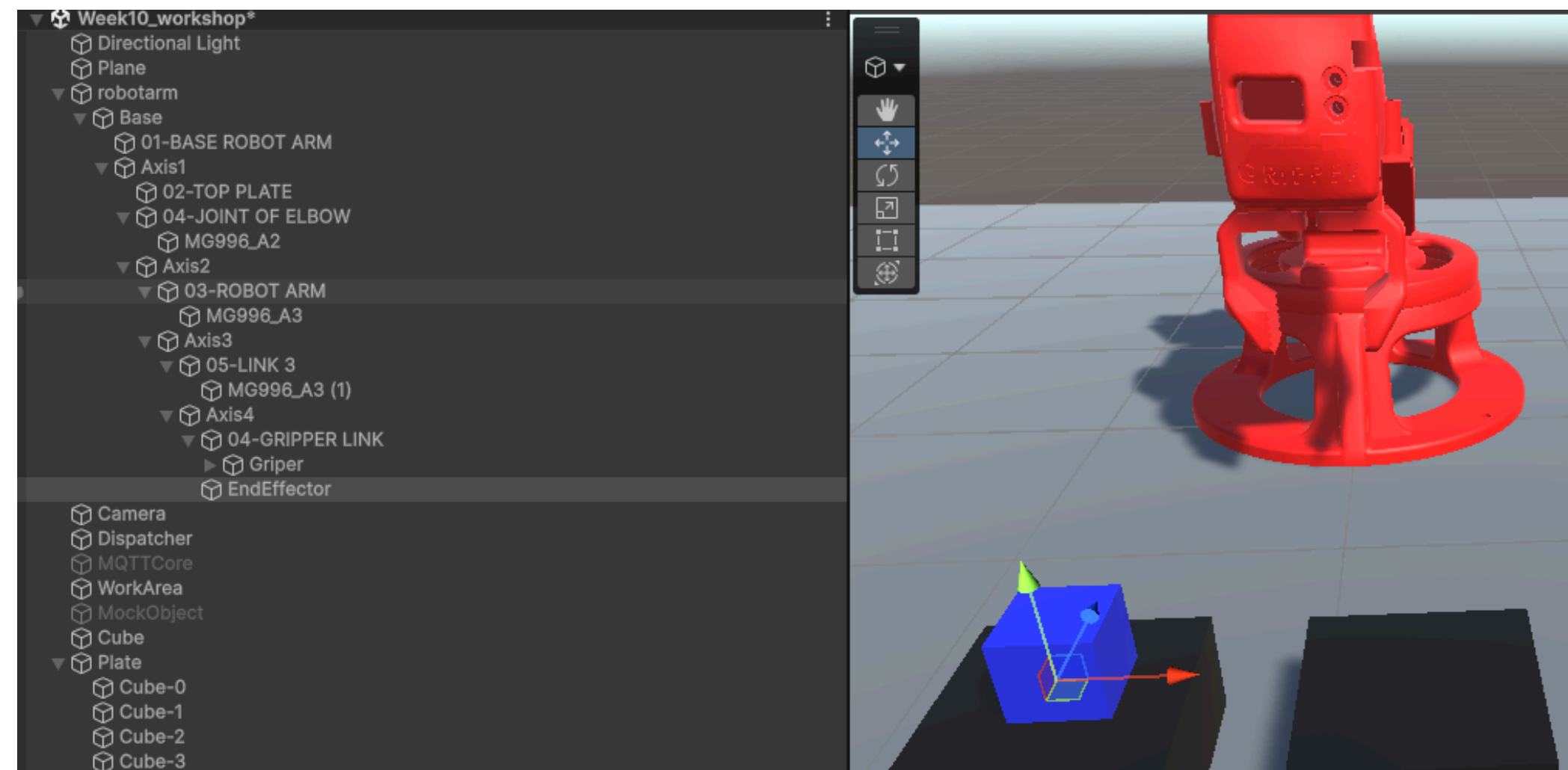
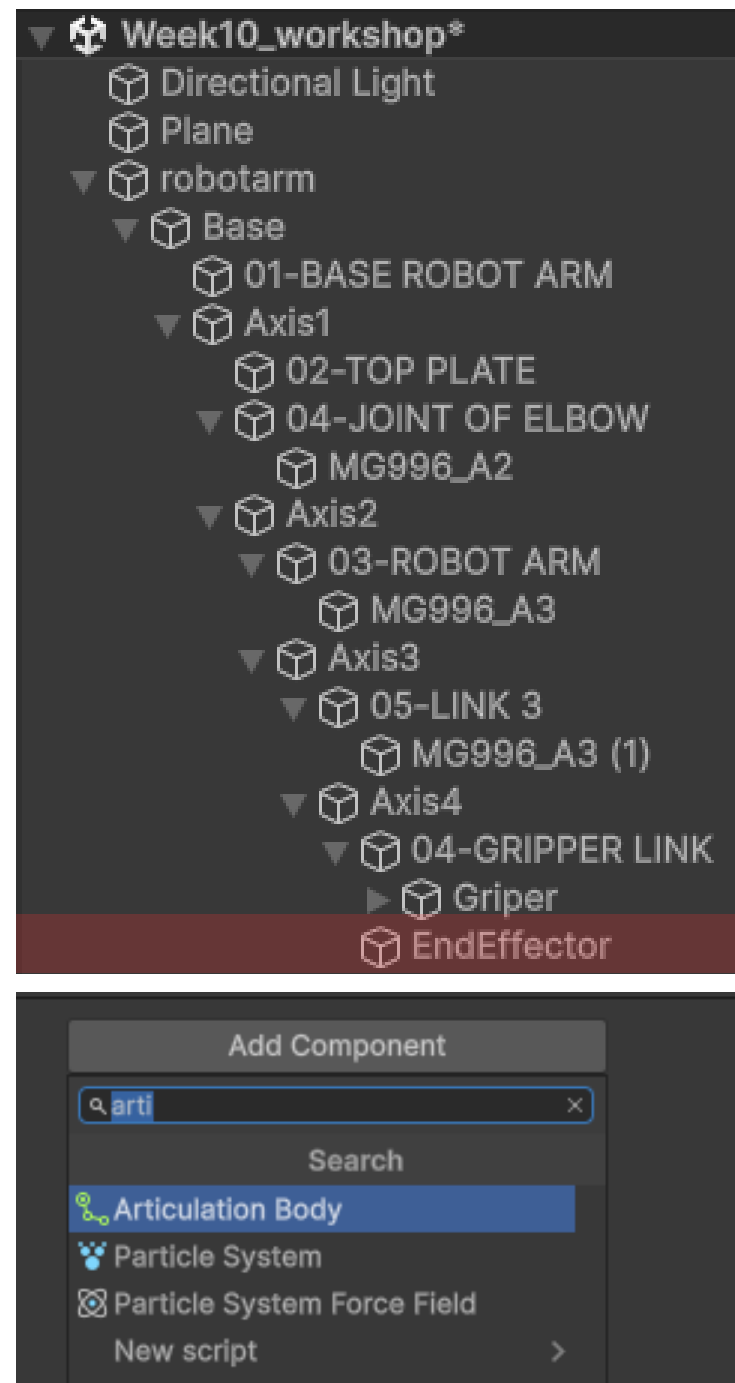
STEP 1 – Hierarchy → Create Empty → ตั้งชื่อ JointTester → Add Component → JointTester



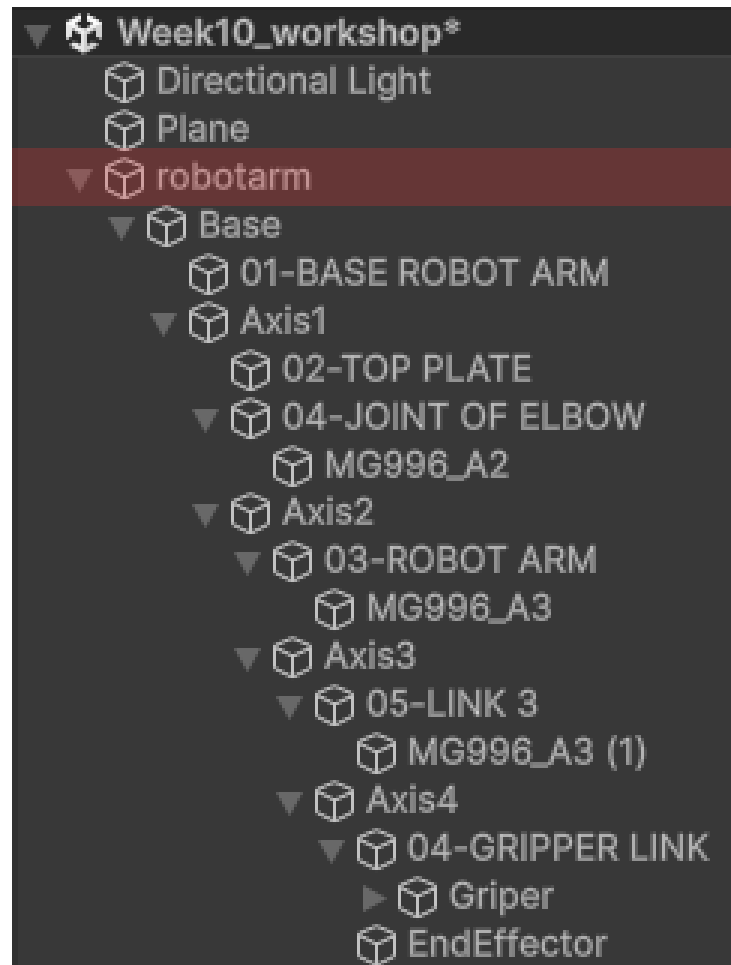
STEP – กด play → เลือก JointTester และปรับ J1-J4 สำหรับเช็คว่ robot arm ขยับถูกต้อง

Step 3: Add markers

STEP — เลือก Axis4 → Create Empty → ตั้งชื่อ EndEffector → ขยับ box ให้อยู่ควบคู่กับ Object.Cube และวางไว้ที่ Plate slot0 (Cube-0)



Step 4: Setup for training robotarm



STEP — เลือก robotarm → Add Component→

- ArmAgent
- Behavior Parameters
- Decision Requester

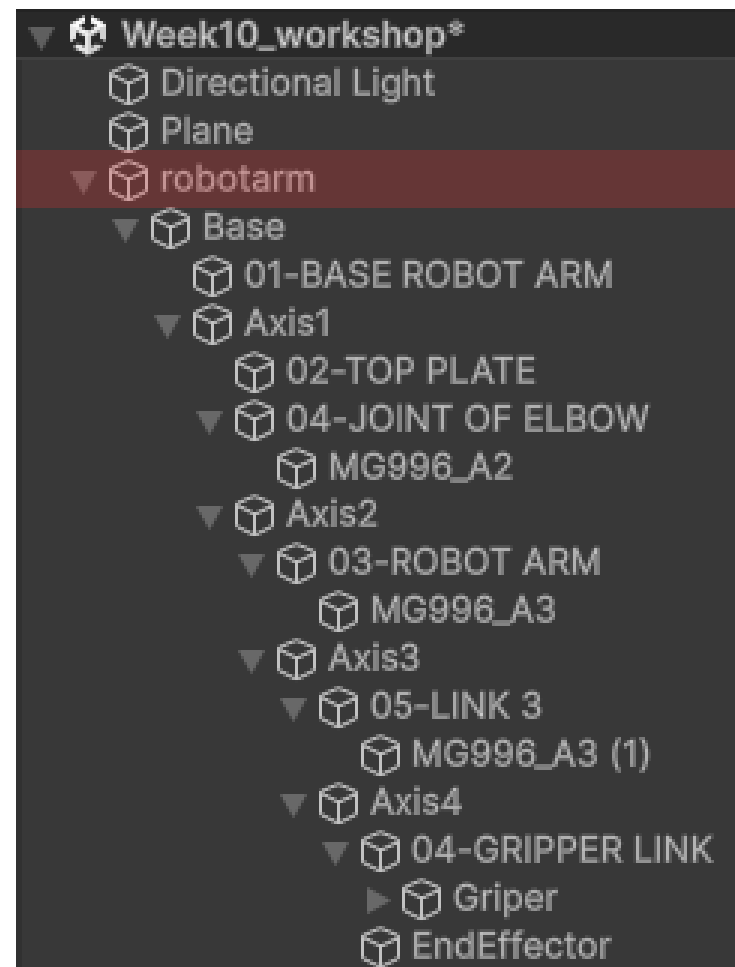
3.2 — Behavior Parameters

Field	Value
Behavior Name	ArmReach
Vector Observation → Space Size	11
Actions → Continuous Actions	4
Model	None (empty)
Behavior Type	Default

3.3 — Decision Requester

- Decision Period = 5 (leave "Take Actions Between Decisions" checked)

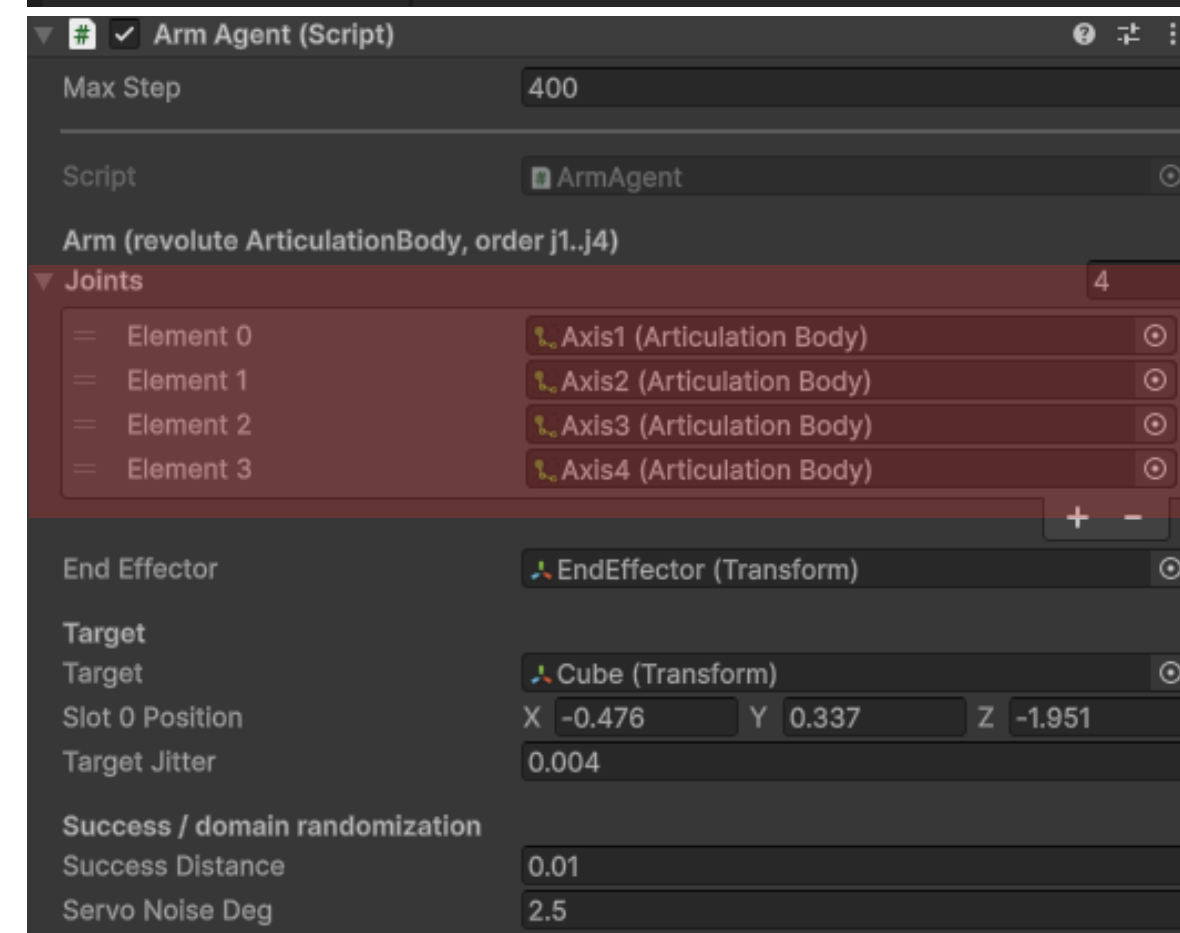
Step 4: Setup robotarm



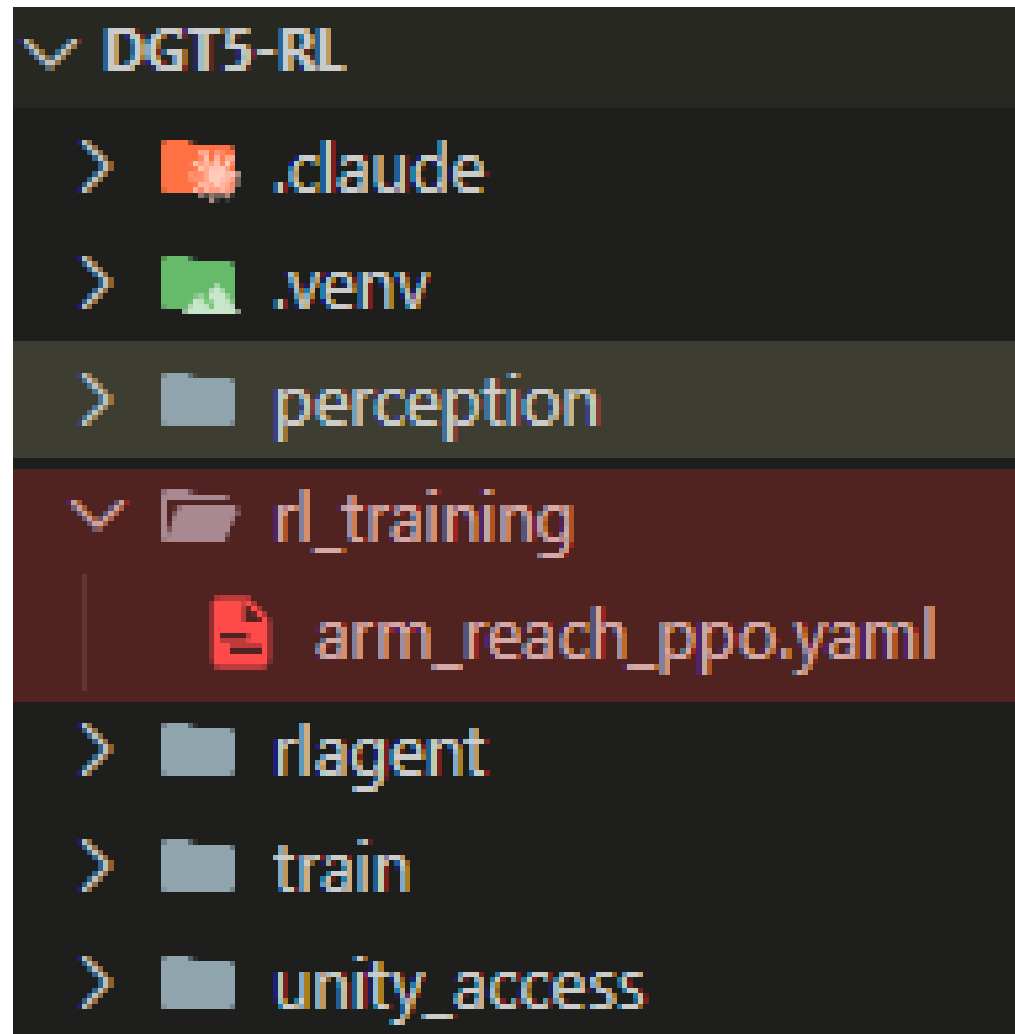
STEP – เลือก robotarm →
Add Component→

- ArmAgent
- Behavior Parameters
- Decision Requester

3.4 — ArmAgent (wire It)	
Field	Value
joints	Size 4 → Axis1, Axis2, Axis3, Axis4 (their ArticulationBodies)
endEffector	EndEffector
target	MockObject
slot0Position	Cube-0's X, Y, Z (optionally +0.01 on Y so the tip stops at object height)
targetJitter	0.004
successDistance	0.01
servoNoiseDeg	2.5
Max Step	400



Step 5: write training config

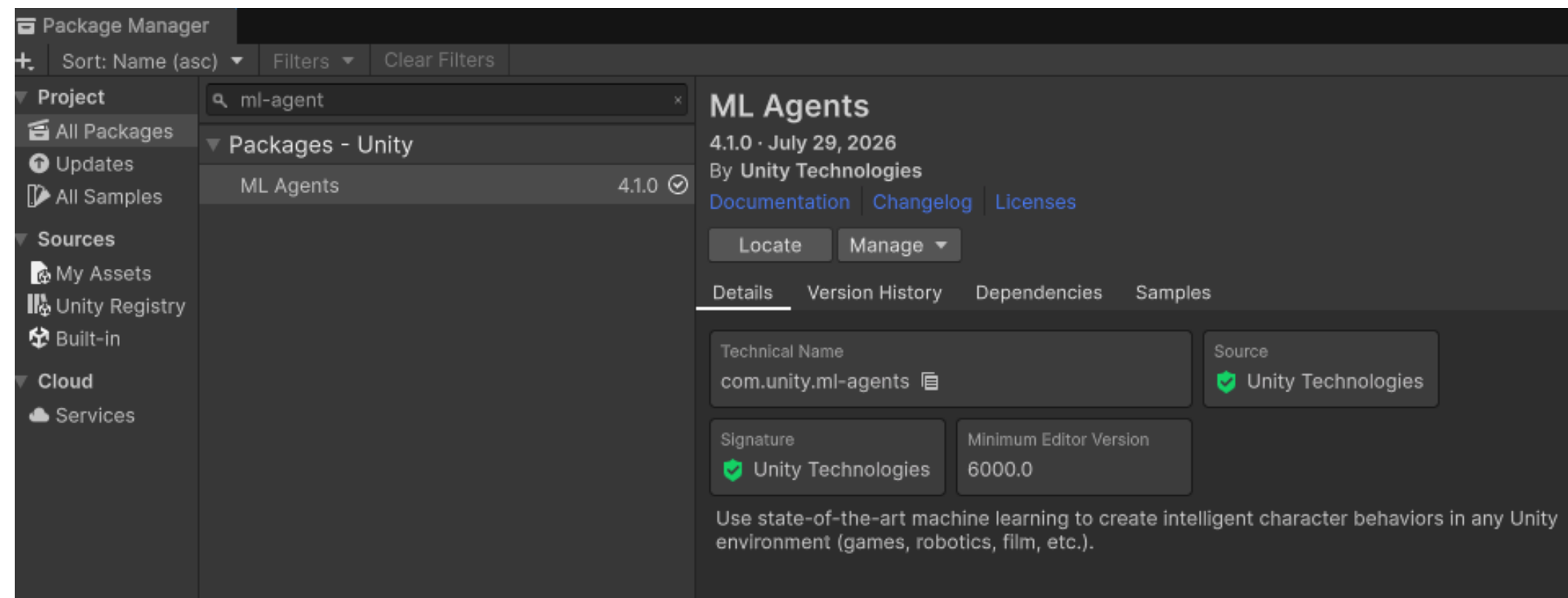


STEP — สร้าง folder “rl_training” → สร้างไฟล์ arm_reach_ppo.yaml

```
1 # PPO config
2 # component "Behavior Name" in the Behavior Parameters in Unity ("ArmReach").
3 # Run: mlagents-learn rl_training/arm_reach_ppo.yaml --run-id=arm_reach_1
4 # then press Play in Unity.
5 behaviors:
6   ArmReach:
7     trainer_type: ppo
8     hyperparameters:
9       batch_size: 1024
10      buffer_size: 10240
11      learning_rate: 3.0e-4
12      beta: 5.0e-3
13      epsilon: 0.2
14      lambda: 0.95
15      num_epoch: 3
16      learning_rate_schedule: linear
17     network_settings:
18       normalize: true
19       hidden_units: 256
20       num_layers: 2
21     reward_signals:
22       extrinsic:
23         gamma: 0.99
24         strength: 1.0
25     max_steps: 2000000
26     time_horizon: 64
27     summary_freq: 10000
```


Step 6: prepare for training

STEP — สร้าง folder “rl_training” → สร้างไฟล์ arm_reach_ppo.yaml



STEP — To add the ML-Agents package to a Unity project:
Open the Package Manager, navigate to Window → Package Manager.
Click + and select Add package by name...
Enter **com.unity.ml-agents** *Click Add to add the package to your project.

Step 7: prepare RL environment

STEP — download python version 3.10.10

www.python.org/downloads/release/python-31010

Install : add Path , Install Python 3.10 for all user

STEP — In VSCODE → new terminal → deactivate .venv

ย้อนกลับ(cd ..) มาจนถึง path “../dgt5-rl”

STEP — create rl-venv

: py --list

: py -3.10 -m venv rlagent

→ activate rlagent

Install package : pip install mlagents

Step 7: prepare RL environment



STEP — In VSCODE Run command to start training RL model

```
cd C:\...\dgt5-rl
```

```
mlagents-learn rl_training/arm_reach_ppo.yaml --run-id=arm_reach_1
```

STEP — Wait until it prints the Unity logo and:

[INFO] Listening on port 5004. Start training by pressing the Play button in the Unity Editor.



STEP — Press ► Play in Unity. Training begins

COMPLETE.

Step 7: prepare RL environment

The error

ModuleNotFoundError: No module named 'pkg_resources'

Fix In your activated env (rlagent):

pip install "setuptools<81"



- **Watch progress**

In the terminal, every 10,000 steps you'll see a line like:

: Step: 10000. Time Elapsed: 30s. Mean Reward: -0.85. Std of Reward: 0.4.

Run again / continue

New run: change the id — --run-id=arm_reach_2 (reusing an id errors).

Continue the same run: add --resume.

Overwrite the same run: add --force.

COMPLETE.

Thank You For Attention

See You Next Time